

ЛЕКЦИЯ 7 из 8

Dedicated Server в среде Linux (Headless)

Блок 3 — Инфраструктура, Linux и Деплой · ~120 мин

Карта занятия:

①

Linux и
Headless-режим

②

Порты, UDP
и основы Bash

③

Живое демо:
сборка и деплой

④

Архитектура
игрового цикла

Мы решили проблему лагов. Но где живёт сервер?

← Лекция 6

← Лекция 6 · Что мы сделали

- ✓ Client-Side Prediction
- ✓ Server Reconciliation
- ✓ Плавное движение при 200ms пинге
- ✓ Тестирование с искусственной задержкой

Новый вопрос

Новый вопрос

- ? Где запустить этот сервер?
- ? Windows дорого и неудобно
- ? Unity Editor — только для разработки
- ? Нужен отдельный серверный процесс

→ Лекция 7: выделенный сервер на Linux



Linux и Headless-режим

Зачем нужен Linux · Headless Unity · Dedicated Server Build Target



Интересный факт

В 2013 году Valve массово перевела Source-серверы (CS, TF2, L4D2) на Linux.

Причина: снижение инфраструктурных расходов на 40%.

С тех пор Linux = стандарт для игровых серверов.

Источник: доклад Valve на GDC 2013 — «Linux Gaming»

Headless Unity: сервер без монитора

Обычный Unity (клиент)

-  Рендер-пайплайн (URP/HDRP)
-  UI Canvas + кнопки
-  AudioListener
-  Камера
-  Ввод с клавиатуры
-  Локальный игрок



Headless Server

-  Сетевая логика
-  Physics (если нужно)
-  GameManager / состояния
-  Нет рендера
-  Нет UI
-  Нет звука · Нет локального игрока

`Application.isBatchMode == true`

Как сервер узнаёт, что он сервер?

```
using FishNet;  
using UnityEngine;  
  
public class ServerAutoStart : MonoBehaviour  
{  
    private void Start()  
    {  
        // true когда Unity запущен без графики  
        if (Application.isBatchMode)  
        {  
            Debug.Log("[Server] Headless mode detected.");  
            InstanceFinder.ServerManager.StartConnection();  
        }  
    }  
}
```

Флаги запуска:

./Server.x86_64 -batchmode -nographics

-batchmode → без UI

-nographics → без рендера

⚠ Без этой проверки — сервер запустится, но будет ждать нажатия UI-кнопки, которой нет.

Dedicated Server Build Target в Unity 2021+

①

Unity Hub

Добавить модуль:
Linux Dedicated Server
Build Support

②

Build Settings

File → Build Settings
→ Dedicated Server
→ Linux
→ Build

③

Результат

GameServer.x86_64
GameServer_Data/
_BackUpThis.../

Dedicated Server Build ≠ Linux Standalone

Клиентский код автоматически вырезается компилятором. Серверный билд меньше и быстрее стартует.

Host/Client vs Dedicated Server — что меняется в коде?

Параметр	Listen Server (Host)	Dedicated Server
Запуск	Кнопка в UI	<code>Application.isBatchMode</code>
Локальный игрок	Есть (Host-клиент)	Нет
Камера	Нужна	Не нужна

Графика Полная `-batchmode -nographics`

Listen Server

A (Host+Client)

↙ ↘

B C

Dedicated Server

[SERVER]

↙ ↓ ↘

A B C



Порты, UDP и основы Bash

Что такое порт · Почему UDP · NAT и Firewall · 8 команд терминала

IP-адрес = дом, порт = номер квартиры

192.168.1.100



IP-адрес = дом

80:

22:

443:

7770:

Порты = двери

Диапазоны портов:

0 – 1023 Системные (HTTP=80, SSH=22)

1024 – 49151 Зарегистрированные

49152 – 65535 Динамические / игровые

FishNet/Tugboat: UDP порт 7770

Один IP-адрес может одновременно обслуживать сотни разных приложений — каждое на своём порту.

TCP vs UDP — почему игры выбирают UDP?

Характеристика	TCP	UDP
Гарантия доставки	✓ Да (ретрансмиссия)	✗ Нет
Порядок пакетов	✓ Гарантирован	✗ Не гарантирован
Задержка	✗ Выше (ожидание)	✓ Минимальная
Потери пакетов	Блокирует очередь	Просто теряются
Подходит для	Файлы, HTTP, чат	Позиции, движение

В шутере устаревшая позиция игрока = мусор. Лучше потерять 1 пакет, чем ждать ретрансмиссии 50ms.

FishNet + Tugboat — UDP с надёжным каналом поверх для важных данных.



Интересный факт: зачем Valve изобрела SDR?

RAW UDP через интернет непредсказуем:
провайдеры могут ограничивать трафик, NAT-конфигурации — самые разные.

В 2014 году Valve выпустила Steam Datagram Relay (SDR) —
кастомный протокол поверх UDP с умной маршрутизацией
через серверы Valve по всему миру.

Результат: CS2 работает стабильнее даже на «плохом» интернете.

NAT, Firewall — почему клиент не может достучаться?

Windows Firewall

Windows (IP: 192.168.1.5)

WSL 2 (IP: 172.25.160.1)

[SERVER:7770]

↑ UDP пакет **БЛОКИРУЕТСЯ** без правила!

⚠ Причина 1: Windows Firewall блокирует UDP 7770

Фикс: `netsh advfirewall firewall add rule
name="GameServer" dir=in action=allow
protocol=UDP localport=7770`

⚠ Причина 2: Неверный IP

`ipconfig` → Windows IP ❌
`hostname -I` → WSL IP ✅

⚠ Причина 3: Сервер слушает 127.0.0.1

Нужно: `0.0.0.0` (все интерфейсы)

WSL 2 — это виртуальная машина внутри Windows

Твой компьютер

Windows IP: 192.168.1.5 (видим из сети)

WSL 2 (виртуальная сеть)

IP: 172.25.160.X (видим только с этого ПК)

Ubuntu + твой сервер :7770

Узнать IP WSL — запускать в терминале WSL!

```
hostname -I
```

Пример вывода: 172.25.160.1

Проверить, слушает ли сервер

```
ss -ulnp | grep 7770
```

✗ Не пиши IP из `ipconfig` (Windows)!

✓ Всегда `hostname -I` в WSL-терминале!

Bash для Unity-разработчика: 8 команд и всё

Команда	Что делает
<code>cd /mnt/e/Buils/GameServer</code>	Перейти в папку с билдом
<code>ls -la</code>	Список файлов с правами доступа
<code>chmod +x ./GameServer.x86_64</code>	Дать права на выполнение файлу
<code>./GameServer.x86_64 -batchmode</code>	Запустить headless-сервер
<code>hostname -I</code>	Узнать IP-адрес WSL
<code>Ctrl + C</code>	Остановить запущенный процесс
<code>cat /proc/cpuinfo</code>	Информация о процессоре
<code>top</code>	Мониторинг процессов (Task Manager)

Этих 8 команд достаточно для полного цикла деплоя сервера в этом курсе.



"Bash — это как C#,
только без автодополнения,
подсветки синтаксиса,
документации и прощения за опечатки.

Зато бесплатно."

Каждый Unity-разработчик, впервые открывший терминал



Живое демо: сборка → запуск → подключение

ServerAutoStart · Build Settings · WSL · Диагностика ошибок

Шаг 1 из 5: Скрипт автозапуска сервера

```
using FishNet;
using FishNet.Managing;
using UnityEngine;

public class ServerAutoStart : MonoBehaviour
{
    [Header("Server Settings")]
    [SerializeField] private ushort port = 7770;

    private void Start()
    {
        // В headless-режиме нет экрана и UI
        if (Application.isBatchMode)
        {
            Debug.Log($"[Server] Starting on port {port}");
            InstanceFinder.ServerManager.StartConnection(port);
        }
    }
}
```

📌 Скрипт кладётся на объект в сцене — больше ничего не нужно.

📌 isBatchMode = true только при флаге -batchmode

📌 В Editor этот блок не выполнится — безопасно для разработки

Шаг 2 из 5: Убрать лишний код из серверного билда

```
// ❌ Плохо: UI-код попадает в серверный билд
private void Start()
{
    startButton.onClick.AddListener(StartServer);
    // Null Reference на dedicated server!
}
```

```
// ✅ Хорошо: вырезаем клиентский код
private void Start()
{
    #if !UNITY_SERVER
        startButton.onClick.AddListener(StartServer);
        mainCanvas.SetActive(true);
    #endif
    #if UNITY_SERVER
        Debug.Log("[Server] Ready.");
    #endif
}
```

UNITY_SERVER определяется автоматически при Dedicated Server Build Target.

Шаг 3 из 5: Build Settings → Dedicated Server → Linux

① **File → Build Settings**

② **Выбрать платформу: Dedicated Server**
(не Linux Standalone!)

③ **Target Platform: Linux**

④ **Добавить нужные сцены в Build**
(минимум: загрузочная + игровая)

⑤ **Нажать Build → Сохранить в папку**

Build Settings

Scenes in Build:

- ☒ Scenes/Boot
- ☒ Scenes/GameScene

Platform: [Dedicated Server] ←

Target OS: Linux ←

[Build]

⚠ В Unity Hub должен быть установлен модуль «Linux Dedicated Server Build Support». Без него Build Target не появится.

Шаг 4 из 5: Что получили и как запустить в WSL

 **GameServer/**

- GameServer.x86_64 ← исполняемый файл
- GameServer_Data/
 - app.info
 - Managed/ ← .dll
 - il2cpp_data/
 - _BackUpThisFolder/

```
# 1. Перейти в папку
$ cd /mnt/e/Builds/GameServer
# 2. Проверить — нет x права!
$ ls -la
# 3. Дать права выполнения
$ chmod +x ./GameServer.x86_64
# 4. Запустить сервер
$ ./GameServer.x86_64 -batchmode
[Server] Starting on port 7770
[Server] Connection ready.
```

Шаг 5 из 5: Получить IP и подключить Windows-клиент

WSL (Ubuntu)

\$ hostname -I IP: 172.25.160.1

→ 172.25.160.1 ← Port: 7770

[Connect]

[SERVER:7770] ← Клиент подключён!

Windows (Unity)

В терминале WSL — узнать IP

\$ hostname -I

172.25.160.1

Проверить что порт слушается

\$ ss -ulnp | grep 7770

udp UNCONN ... *:7770

В клиенте (Unity Editor / билд):

1. Открыть UI подключения
2. Ввести IP из hostname -I
3. Порт: 7770

4. Нажать Connect

ТОП-3 ошибки на Практике 4

❌ Ошибка 1: Сервер не стартует автоматически

Симптом: нет логов «Connection ready»

Причина: Application.isBatchMode не проверяется

```
if (Application.isBatchMode) {  
    InstanceFinder.ServerManager.StartConnection(port);  
}
```

❌ Ошибка 2: Клиент не подключается по IP

Причина: используется IP из ipconfig вместо hostname -I

ipconfig → 192.168.1.5 ❌ Windows IP

hostname -I → 172.25.160.1 ✅ WSL IP

❌ Ошибка 3: Windows Firewall блокирует UDP

Фикс (PowerShell от администратора):

```
netsh advfirewall firewall add rule name="GameServer"  
dir=in action=allow protocol=UDP localport=7770
```

Алгоритм диагностики: клиент не подключается

① Запущен ли сервер?

→ В WSL-консоли видны логи «Connection ready»?

② Правильный ли IP?

→ hostname -I в WSL, не ipconfig в Windows

③ Слушает ли сервер нужный порт?

→ ss -ulnp | grep 7770

④ Пропускает ли Firewall?

→ Создать правило netsh (см. пред. слайд)

⑤ Правильный ли порт в клиенте?

→ NetworkManager → Transport → Port = 7770

⑥ Стартует ли сервер автоматически?

→ Проверить isBatchMode в коде

4

Архитектура игрового цикла

Машина состояний · WaitingForPlayers → InProgress → ShowingResults

Сервер — дирижёр всего матча

WaitingForPlayers

старт сервера

N игроков подключились

InProgress

матч идёт

таймер или счёт

ShowingResults

пауза 5 сек, повтор



Только сервер управляет переходами

Клиенты реагируют через SyncVar OnChange

```
public enum MatchState
{
    WaitingForPlayers,
    InProgress,
    ShowingResults
}
```

Золотое правило: клиент реагирует, а не создаёт состояние

[Server] GameState (SyncVar) — OnValueChanged — авторассылка FishNet

[Client A] обновить UI [Client B] обновить UI [Client C] обновить UI

❌ Плохо

```
// Плохо: клиент сам решает
void Update() {
    if (connectedPlayers >= 2)
        StartMatch();
    // каждый клиент считает по-своему!
}
```

✅ Правильно

```
// Правильно: только сервер решает
[SyncVar(OnChange = nameof(OnStateChanged))]
private MatchState gameState;

void OnStateChanged(MatchState prev,
    MatchState next, bool asServer) {
    if (next == MatchState.InProgress)
        ShowMatchUI();
}
```



Интересный факт: как Valve управляет матчами

CS2-серверы Valve работают без перезагрузки неделями и месяцами.

Между раундами выполняется:

`mp_restartgame 1`

Это сбрасывает состояние матча ВНУТРИ работающего процесса.

Никакого restart сервера. Никакого переподключения игроков.
Просто сброс переменных.

Именно это мы реализуем в Практике 4.

Перезапуск сервера = плохая архитектура

Restart сервера между раундами

- ❌ Холодный старт: 2–5 секунд
- ❌ Все клиенты теряют соединение
- ❌ Клиенты должны переподключаться
- ❌ Потеря статистики сессии
- ❌ Видимый лаг в UX

Правильный подход

- ✅ Сервер работает непрерывно
- ✅ Сброс: score = 0, timer = 60
- ✅ Respawn всех игроков
- ✅ GameState → WaitingForPlayers
- ✅ Игроки не замечают разрыва

На Практике 4 реализуем именно правый вариант.
Сервер запускается один раз и крутит циклы бесконечно.

Практика 4: что вам предстоит сделать

Что нужно реализовать:

- ① Установить WSL 2 + Ubuntu
- ② Добавить Linux Server Build Support в Unity Hub
- ③ Написать ServerAutoStart.cs
- ④ Собрать Dedicated Server билд под Linux
- ⑤ Запустить в WSL: chmod +x, запуск
- ⑥ Подключить Windows-клиент по IP
- ⑦ Реализовать GameManager с тремя состояниями
- ⑧ Лобби с отображением 1/2 игроков
- ⑨ Автостарт матча при достижении лимита
- ⑩ Экран результатов и возврат в лобби

Ожидаемый результат:

 Консоль WSL показывает серверные логи

 Windows-клиент подключается к Linux-серверу

 **Полный цикл без остановки:**
Лобби → Матч → Результаты → Лобби

Что дальше → Лекция 8: Matchmaking и финализация

✓ Лекция 7 — итоги

- ✓ Linux vs Windows: экономика
- ✓ Headless-режим и isBatchMode
- ✓ Dedicated Server Build Target
- ✓ UDP, порты, NAT, Firewall
- ✓ 8 Bash-команд для деплоя
- ✓ Машина состояний матча
- ✓ Сервер управляет циклом

→ Лекция 8

Matchmaking и финализация

- 📌 Глобальные свойства комнаты
- 📌 Жизненный цикл сессии полностью
Лобби → Игра → Результаты
- 📌 Оптимизация билда для релиза
- 📌 Финальная сборка проекта

Что мы узнали — Лекция 7

① Linux в GameDev

- Бесплатный Ubuntu vs платный Windows Server
- 200–400 MB RAM vs 1.5–2 GB
- 80–90% серверов в индустрии на Linux

② Headless-режим

- Application.isBatchMode — ключевой флаг
- Флаги: -batchmode -nographics
- #if UNITY_SERVER — условная компиляция
- Dedicated Server Build Target

③ Сеть и Bash

- Порт = номер квартиры (FishNet: 7770/UDP)
- UDP: актуальность важнее надёжности
- WSL 2 имеет собственный IP (hostname -I)
- 8 команд: cd, ls -la, chmod +x, top...

④ Архитектура цикла

- 3 состояния: Waiting → InProgress → Results
- Сервер управляет переходами (SyncVar)
- Клиенты реагируют, не создают состояние
- Нет перезапуска сервера между раундами